

Ejercicio: Simulación del Ejército Romano en JavaScript (ES6 + Módulos)

1. Introducción Histórica

El ejército romano, a lo largo de sus más de mil años de historia, cambió su estructura, tamaño y organización. Roma no se construyó en un día y atravesó tres grandes periodos:

1.1. Monarquía

Los romanos se deshicieron de sus reyes, aunque siguieron honrando a sus fundadores, Rómulo y Remo.

1.2. República

La República implantó un sistema donde los ciudadanos elegían sus cargos. Durante este periodo surgieron fuertes tensiones entre:

- **Populares:** defendían otorgar ciudadanía a poblaciones conquistadas.
- **Optimates:** ricos terratenientes que se oponían.

Los enfrentamientos fueron violentos, con asesinatos políticos como los de los hermanos Graco.

El caos terminó temporalmente bajo **Sila**, que se autoproclamó dictador vitalicio, aunque luego renunció.

Tras nuevas guerras internas, **Julio César** cruzó el Rubicón, se enfrentó al Senado y se convirtió en dictador. Finalmente fue asesinado por su hijo adoptivo Bruto y otros conspiradores.

1.3. Imperio

Aunque las guerras internas continuaron, el Imperio logró grandes conquistas, alcanzando su extensión máxima bajo **Traiano**.

Después de la dinastía Severa comenzó la decadencia, con enfermedades como la Peste Antonina, crisis agrícolas y presiones migratorias desde Asia Central, entre ellos los hunos de Atila. **El Imperio Occidental cayó, mientras que el Oriental sobrevivió hasta el siglo XV.**

2. Descripción Histórica del Ejército (por Hendrik Pahl)

Se conserva el texto original citado del historiador Hendrik Pahl, omitido aquí para no duplicarlo. Incluye estructura de la legión, número de hombres, tipos de tropas y organización interna.

3. Simplificación para el Ejercicio

Vamos a crear una **simulación simplificada** en JavaScript de parte del ejército romano, centrada en:

- Soldados.
- Centurias.
- Legiones.

Simplificaciones aplicadas:

- Cada **centuria** tendrá solo **10 soldados**, no 80–100 como históricamente.
- La legión podrá tener un máximo de **10 centurias**.
- Solo **una unidad de caballería (equites)** por legión.
- El número de **velites** (infantería ligera) no puede superar la **mitad** del número de centurias normales.
- Cada clase irá en un archivo **módulo JS separado**.

4. Especificación de Clases

4.1. Clase **Soldado**

Constructor

`constructor(nombre, edad, tipo)`

Atributos privados

- `#_nombre` (String)
 - Mínimo 5 caracteres
 - Lanza excepción si no se cumple
- `#_edad` (Number)
 - Entero entre 13 y 84
 - Lanza excepción si no se cumple
- `#_tipo` (String)
 - Debe pertenecer al array inmutable { `centurio`, `velite`, `equite` }
 - Lanza excepción si no se cumple
- `#_provincia` (String)

Métodos

- **Getters y Setters ES6** (publicados como `get propiedad()` / `set propiedad()`)
- `toString()`
 - Muestra todos los atributos
 - Incluye *una imagen (emoji/ASCII)* representativa del soldado

Ejemplo

```
let publio = new Soldado("Marco Quinto", 30, equite);
```

4.2. Clase Centuria

Constructor

`constructor(codigo, tipo)`

Atributos privados

- `#_codigo` (Number)
- `#_tipo` (String) — { `centurio`, `velite`, `equite` }
- `#_tropa` (Set) — Máx. 10 soldados
- `#_centurion` — Un único soldado con rol centurio

Métodos

- `addSoldado(soldado)`: añade un soldado
- `delSoldado(nombre, provincia)`: elimina coincidencia exacta
- `asignaCenturion(soldado)`: asigna o reemplaza el centurión jefe
- `toString()`: muestra todos los datos y recorre el Set con técnicas vistas en clase
- Getters y Setters ES6

Ejemplo

```
let centuria1 = new Centuria(1, velite);
```

4.3. Clase **Legion**

Constructor

`constructor(numero, legatus, comentarioHistorico)`

Atributos privados

- `#_numero` (1–30)
- `#_legatus` (String, no vacío)
- `#_comentarioHistorico` (String)
- `#_centurias` (Array) — Solo de tipo centurio
- `#_velites` (Set) — Solo tipo velite
- `#_equite` (Centuria única de caballería)

Métodos

- `addCenturia(centuria)`
 - Solo si es tipo centurio
 - Devuelve `true` o `false`
- `delCenturia(codigo)`
 - Devuelve `true` o `false`
- `addVelite(velite)`
 - Solo tipo velite
 - Devuelve `true` o `false`
- `delVelite(codigo)`
 - Devuelve `true` o `false`
- Asignación de equite (unidad única)
- `toString()`: muestra todo el contenido
- Getters y Setters ES6

Ejemplo

```
let quintaLegion = new Legion(8, "Julio César", "Legio VII Gemina");
```

5. Función de Uso General

Debes crear una función: `function uso();` Dentro de esta:

- Crear **20 soldados** de distintos tipos.
- Crear:
 - 3 centurias normales
 - 2 centurias velite
 - 1 centuria equite
- Crear **2 legiones**.
- Usar **todos los métodos** de todas las clases.
- Reconocer si existe un patrón de diseño adecuado para "crear objetos en cadena".

6. Notas y Reglas Importantes

Sobre código y estilo

- Todo en **ES6**.
- Usar **privados** con `#_propiedad`.
- Prohibido:
 - patrones de diseño con múltiples returns
 - funciones flecha en clases
 - variables globales
 - `while(true)`
 - `continue`
 - `break` fuera de `switch`
 - expresiones regulares
 - `const` excepto para arrays inmutables
 - `document.write()`
- A partir de unidad 3: no mostrar datos desde métodos no E/S.
- Siempre incluir **default** en `switch`.

Buena práctica obligatoria

- Comentarios explicativos en funciones, parámetros y bloques clave.
- Variables y constantes con nombres claros.
- Finalizar sentencias con `“;”`.

Uso de módulos

- Cada clase en un archivo separado
- La función `uso()` en otro archivo principal.

Salida en pantalla

Ejemplo:

```
<p id="salida"></p>
```

```
<script>  
  salida.innerHTML = "Texto...";  
  salida.innerHTML += objeto.toString();  
</script>
```

Control de excepciones

Cada creación de objeto o uso de setter que puede fallar debe envolverlo en:

```
try {  
  ...  
} catch (e) {  
  console.error("Error: " + e);  
}
```

7. Resumen Final

En esta tarea debes:

- Modelar soldados, centurias y legiones con clases ES6.
- Usar módulos, privados, sets y arrays.
- Respetar todas las restricciones de estilo y control.
- Simular la creación y organización del ejército.
- Aplicar manejo de excepciones en todas las operaciones críticas.